

Metodología de Sistemas 2

Trabajo Práctico Unidad 2

Patrones de diseño

Profesora: Ing. Vargas, Carolina

Equipo: Los Metódicos

Integrantes:

- Del Grecco, Daiana
- Gallardi, Angelo

Nombre del patrón seleccionado

Singleton

Tipo de patrón:

De creación (o Creacional)

Descripción del problema que resuelve:

El patrón Singleton resuelve dos problemas al mismo tiempo, vulnerando el Principio de responsabilidad única:

1. **Garantizar que una clase tenga una única instancia.** ¿Por qué querría alguien controlar cuántas instancias tiene una clase? El motivo más habitual es controlar el acceso a algún recurso compartido, por ejemplo, una base de datos o un archivo.

Funciona así: imagina que has creado un objeto y al cabo de un tiempo decides crear otro nuevo. En lugar de recibir un objeto nuevo, obtendrás el que ya habías creado.

2. **Proporcionar un punto de acceso global a dicha instancia.** Las variables globales son muy útiles, pero también son poco seguras, ya que cualquier código podría sobrescribir el contenido de esas variables y descomponer la aplicación.

Al igual que una variable global, el patrón Singleton nos permite acceder a un objeto desde cualquier parte del programa. No obstante, también evita que otro código sobreescriba esa instancia.

Descripción de solución

¿Cómo hace Singleton para asegurar que una clase tenga una única instancia y que ésta sea fácilmente accesible? Una variable global hace accesible a un objeto, pero no nos previene de crear múltiples instancias de objetos.

La solución que propone Singleton es hacer que sea la propia clase la responsable de su única instancia. La clase puede garantizar que no se puede crear ninguna otra instancia (interceptando las peticiones para crear nuevos objetos), y puede proporcionar un modo de acceder a la instancia.

Participantes

Singleton:

- define una operación Instancia que permite que los clientes accedan a su única instancia. Instancia es una operación de clase (es decir, un método de clase en Smalltalk y una función miembro estática en C++).
- puede ser responsable de crear su única instancia.

Colaboración

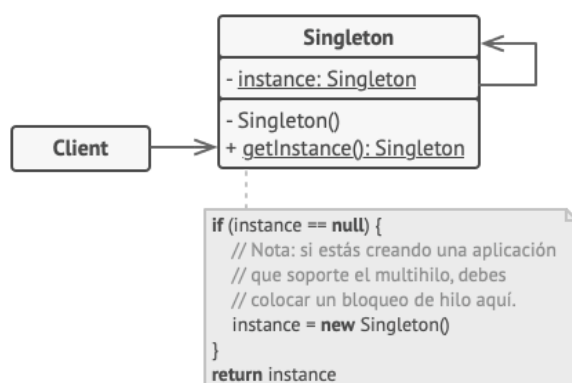
Los clientes acceden a la instancia de un Singleton exclusivamente a través de la operación Instancia de éste.

Consecuencias

El patrón Singleton proporciona varios beneficios:

1. Acceso controlado a la única instancia. Puesto que la clase Singleton encapsula su única instancia, puede tener un control estricto sobre cómo y cuándo acceden a ella los clientes.
2. Espacio de nombres reducido. El patrón Singleton es una mejora sobre las variables globales. Evita contaminar el espacio de nombres con variables globales que almacenen las instancias.
3. Permite el refinamiento de operaciones y la representación. Se puede crear una subclase de la clase Singleton, y es fácil configurar una aplicación con una instancia de esta clase extendida. Podemos configurar la aplicación con una instancia de la clase necesaria en tiempo de ejecución.
4. Permite un número variable de instancias. El patrón hace que sea fácil cambiar de opinión y permitir más de una instancia de la clase Singleton. Además, podemos usar el mismo enfoque para controlar el número de instancias que usa la aplicación. Sólo se necesitaría cambiar la operación que otorga acceso a la instancia del Singleton.
5. Más flexible que las operaciones de clase. Otra forma de empaquetar la funcionalidad de un Singleton es usar operaciones de clase (es decir, funciones miembro estáticas en C++ o métodos de clase en Smalltalk). Pero ambas técnicas de estos lenguajes dificultan cambiar un diseño para permitir más de una instancia de una clase. Además, las funciones miembro estáticas en C++ nunca son virtuales, por lo que las subclases no las pueden redefinir polimórficamente.

Ejemplo



La clase Singleton declara el método estático `obtenerInstancia` que devuelve la misma instancia de su propia clase.

El constructor del Singleton debe ocultarse del código cliente. La llamada al método `obtenerInstancia` debe ser la única manera de obtener el objeto de Singleton.

Bibliografía

- Shvets, A. (s.f.). *Singleton*. Refactoring.Guru. Recuperado el 25 de agosto de 2026, de <https://refactoring.guru/es/design-patterns/singleton>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (2002). *Patrones de diseño: Elementos de software orientado a objetos reutilizable*. Pearson Educación.

El Patrón Singleton: Una Sola Instancia, Control Total

El Problema: Desafío y Riesgo



El Desafío de la Instancia Única
Se requiere evitar la creación múltiple de objects para proteger recursos compartidos críticos.



Acceso Global vs. Seguridad
Ofrece acceso desde cualquier parte del programa sin los riesgos de sobrescritura de variables globales.

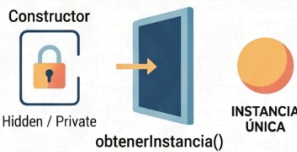


**La Solución:
Singleton y su Control**
La clase intercepta peticiones de creación y devuelve siempre la instancia ya existente.



Implementación y Ventajas

Método obtenerInstancia



Espacio de Nombres Limpio
Evita contaminar el código con variables global organizando mejor el acceso a los datos.



La Clase como su propio Guardián
La lógica interna de la clase gestiona el ciclo de vida del objeto para asegurar su exclusividad



Flexibilidad y Refinamiento
Permite crear subclases o cambiar a un número controlado de instancias fácilmente.

Comparación: Singleton vs. Variable Global

	Singleton	Variable Global
 Instancia Única	 Garantizada por la clase	 No se puede garantizar
 Seguridad	 Protegida contra sobrescritura	 Vulnerable a cambios externos
 Organización	 Encapsulada en un espacio propio	 Contamina el espacio de nombres